Mikrodatorteknik (TMIK13) – Tenta B med svar

Fil: GetInsperaExamPdf 1.pdf · 14 uppgifter · 50 p

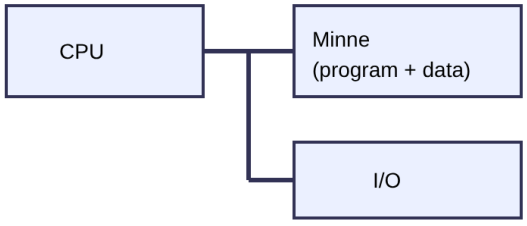
> **Hur du läser:** Varje uppgift har först **Svar**, sedan en kort *Förklaring* på enkel svenska. Skriv gärna egna kommentarer i filen.

1. Harvard och von Neumann (2 p)

Rita ett blockschema för Harvard resp. von Neumann.

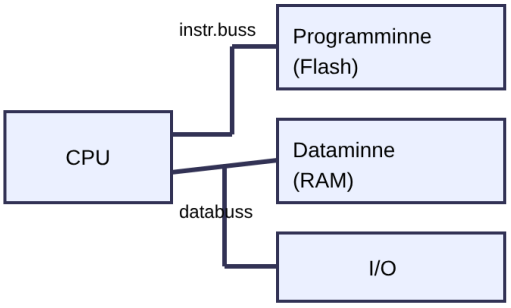
Svar:

von Neumann (en buss för allt)



En gemensam adress-/databuss.
CPU kan bara hämta instruktion
ELLER data åt gången (flaskhals).

Harvard (separata bussar)



Instruktion och data kan hämtas
samtidigt (snabbare). STM32/ARM Cortex-M = Harvard.

- **von Neumann:** program och data ligger i **samma minne** och delar **en buss**. Enkelt men CPU:n kan inte hämta instruktion och data samtidigt (“von Neumann-flaskhalsen”).
- **Harvard: separata minnen och bussar** för program (instruktioner) och data. CPU:n kan hämta nästa instruktion samtidigt som den läser/skriver data → snabbare. ARM Cortex-M (STM32) bygger på Harvard-principen.

Tips på tentan: Rita rutor för CPU, minne(n) och I/O, och visa tydligt **en** buss vs **två** bussar.

2. Pull-up / pull-down (2 p)

Svar:

- **Pull-up:** ett motstånd mellan pinnen och **VCC (3.3 V)**. När inget annat driver pinnen läses den som **1 (hög)**.
- **Pull-down:** ett motstånd mellan pinnen och **GND**. När inget annat driver pinnen läses den som **0 (låg)**.

När används det? När en ingång annars skulle "flyta" (vara odefinierad), t.ex. en knapp som bara kopplar till GND (då behövs pull-up), eller en sensor med **open-drain/open-collector**-utgång som bara kan dra ner till GND. Även I2C-linjer behöver pull-up.

Hur aktiveras det? Antingen med ett yttre motstånd (typ 4.7–10 kΩ) eller med det **inbyggda** motståndet i STM32: registret **PUPDR** (i CubeMX: GPIO → "Pull-up" / "Pull-down").

Förklaring: En ingång som inte är kopplad till något är som en antenn – den kan läsas som 0 eller 1 slumpmässigt. Motståndet ger ett "standardläge" men är så stort att knappen/sensorn lätt kan ändra nivån.

3. ADC – strömsensor (1 p)

VOUT = 0 V vid 0 A, +0.2 V per ampere. Digitalt värde 3278 – vilken ström?

Svar: ≈ 13.2 A

Förklaring (2 steg):

1. Värde → spänning: U = 3278 / 4095 × 3.3 = **2.642 V**
2. Spänning → ström: I = 2.642 V / 0.2 V/A = **13.21 A**

Grundformel ADC: **U = värde / 4095 × 3.3 V** (vissa lärare använder 4096 – ger nästan samma svar).

4. RAM-minne (3 p)

Figuren: 32-pin PDIP med A0–A17 och DQ0–DQ7.

Fråga	Svar
Adresspinnar	**18** (A0–A17)
Ordlängd	**8 bitar** (DQ0–DQ7)
Antal byte	**262144**

Förklaring:

- Räkna alla **A**-pinnar: A0…A17 = 18 st (de står i blandad ordning på kretsen – räkna noga, A17 och A14 sitter uppe till höger).
- Räkna **DQ**-pinnarna (data): DQ0…DQ7 = 8 st → 1 byte per adress.
- 2^18 = **262 144** adresser × 1 byte = **262 144 byte** (= 256 kB).
- RESET#, WE#, OE#, CE#, VCC, VSS är styr-/matningspinnar, inte adress/data.

5. Funktion add_mul (2 p)

```
float add_mul(float a) { return 2/a + 1; }
float b = add_mul(0.4);
```

Svar: b = 6.0 · Flytta till annan .c-fil: **Falskt** (b får inte samma värde).

Förklaring:

- 2 / 0.4 = 5, plus 1 = **6**.
- Om funktionen flyttas till en annan .c-fil **utan** att man lägger till en prototyp (`float add_mul(float a);` i en .h-fil) så vet inte main.c hur funktionen ser ut. Gammal C antar då att den returnerar **int** och skickar 0.4 som **double**. Funktionen tror den får en float och returnerar en float i ett float-register → **b blir skräp/fel värde**. (Nyare kompilatorer ger varning eller fel "implicit declaration".)
- *Lärdom:* lägg alltid prototypen i en header-fil och `#include` den.

6. DMA (2 p)

Svar: DMA = **Direct Memory Access**.

En DMA-styrenhet kan flytta data **utan att CPU:n gör jobbet**: mellan kringenhet ↔ minne (t.ex. ADC → RAM, RAM → UART/SPI) eller minne ↔ minne. CPU:n ställer bara in källa, destination, antal och sedan sköter DMA:n överföringen. När den är klar kan den ge ett avbrott (t.ex. "transfer complete", även "half transfer").

Användningsområden:

- ADC som samplar många värden kontinuerligt → DMA lägger dem i en buffert i RAM.
- Skicka/ta emot mycket data på UART, SPI, I2C (t.ex. en display, SD-kort).
- Ljud (DAC/I2S) där data måste komma i jämn takt.
- Kopiera stora minnesblock.

Fördel: CPU:n blir avlastad – kan göra annat eller sova (spar ström), och överföringen går snabbt och i jämn takt.

Förklaring: Tänk dig DMA som en "flyttgubbe": CPU:n säger "flytta 100 värden från ADC till den här arrayen" och kan sedan göra annat.

7. NVIC (1 p)

Svar: NVIC = **Nested Vectored Interrupt Controller**.

Förklaring: Det är den del i ARM Cortex-M som hanterar alla avbrott (interrupts). "Nested" = ett avbrott med högre prioritet kan avbryta ett avbrott med lägre prioritet. "Vectored" = varje avbrott har en egen adress i en tabell (vektortabellen), så CPU:n hoppar direkt till rätt ISR.

8. GPIO-register (3 p)

Svar:

- **MODER** (mode): väljer läge för varje pinne – **Input, Output, Alternate function** (t.ex. UART, TIM, I2C) eller **Analog** (ADC/DAC). 2 bitar per pinne.
- **OTYPER** (output type): **push-pull** eller **open-drain** för utgången.
- **OSPEEDR** (output speed): hur snabbt utgången kan slå om (low/medium/high/very high). Högre = snabbare flanker men mer störningar/ström.
- **PUPDR** (pull-up/pull-down): slår på intern **pull-up**, **pull-down** eller ingen.
- **IDR** (input data register): **läsa** pinnens nivå (0/1).
- **ODR** (output data register): **skriva** vad utgången ska ge (0/1).

Minnesregel: MODE = vad är pinnen, TYPE = hur driver den, SPEED = hur snabbt, PUPD = motstånd, I = in, O = ut.

9. Assembler (3 p)

```
my_function:
  MOVS R0, #127
0x8001644:
  NOP
  NOP
  SUBS R0, R0, #2
  BNE 0x8001644
```

Fråga	Svar
R0 initialt	**127**
SUBS-instruktionen	**Subtraherar 2 från R0 och uppdaterar flaggor**
BNE beror på	**Zero**
Antal NOP	**Oändligt (loopen blir aldrig klar)**
När R0 “når” 0	**Programmet loopar i oändlighet**
Syfte	**Åstadkomma en kort fördröjning (men har en bugg)**

*Förklaring – det här är en **lurig** fråga:*

- 127 är **udda**. Vi drar bort 2 varje gång: 127, 125, …, 3, **1**, och sedan 1 − 2 = **−1** (= 0xFFFFFFFF). R0 hoppar alltså **över** 0!
- BNE hoppar så länge resultatet inte är 0 (Z = 0). Eftersom R0 aldrig blir exakt 0 (alla värden förblir udda), blir det en **oändlig loop**.
- Det är buggen: tanken var en fördröjning, men startvärdet borde varit jämnt (t.ex. 128) eller man borde dra bort 1.
- (Frågan skriver “SUBS R0, R0, #1” i texten men koden har #2 – välj alternativet med 2.)

10. PLL (2 p)

Svar: PLL = **Phase-Locked Loop** (faslåst slinga). Den används för att **multiplicera upp** en låg klockfrekvens till en högre systemklocka.

Hur den fungerar: En VCO (spänningsstyrd oscillator) genererar en hög frekvens. Den delas ner och jämförs i en **fasdetektor** med referensklockan. Skillnaden filtreras och styr VCO:n så att den låser sig på exakt rätt frekvens.

I STM32L433:

- Klockkälla: HSI16 (16 MHz), MSI eller HSE (extern kristall).
- Först delas den med **M**, sedan multipliceras med **N** (i VCO:n), sedan delas med **R** (systemklocka), **P** och **Q** (andra utgångar, t.ex. USB/SAI).
- f_ut = f_in / M × N / R. Exempel: 16 MHz / 1 × 10 / 2 = **80 MHz** (max för L433).

Förklaring: Kristaller och interna oscillatorer finns bara för låga frekvenser, men CPU:n vill gå snabbt. PLL:en "multiplicerar" klockan och ser till att den ändå är lika stabil som referensen.

11. Sant/falskt om timers (3 p)

Svar (bocka i dessa – de är SANNA):

- ✔ Vissa timers kan räkna även när CPU sover, om deras klockkälla är aktiv. *(t.ex. LPTIM, RTC)*
- ✔ Auto-reload registret (ARR) bestämmer vid vilket värde timern återgår till 0.
- ✔ När räknaren slår om kan en interrupt genereras om den är aktiverad. *(update event)*

FALSKA (bocka inte):

- ✘ "En timer har alltid samma frekvens som CPU-kärnan" – nej, den kan ha annan klocka och delas med prescalern.
- ✘ "Prescalern används för att öka counterns räkningshastighet" – nej, prescalern **delar** klockan → räknaren går **långsammare**.
- ✘ "PWM kan generera en analog spänning direkt" – nej, PWM är bara 0/3.3 V-pulser. Man behöver ett lågpassfilter (RC) för att få en analog medelspänning.

Förklaring: Felbockning ger −1, så bocka bara det du är säker på.

12. Timer-avbrott (1 p)

PSC = 624, 80 MHz, interrupt var 1 ms (1000 ggr/s).

Svar: ARR = 127

Förklaring:

- Timerklocka = 80 MHz / (624 + 1) = **128 kHz** = 128 000 tick/s.

- 1 ms → 128 000 × 0.001 = 128 tick → **ARR = 128 − 1 = 127**.

13. PWM-styrning (5 p)

Givet: 80 MHz, PWM ≈ 10 kHz, 1000 steg.

Register	Svar
PSC	**7**
ARR	**999**
CCR1 (35 %)	**350**
PSC för max frekvens	**0**
Frekvens	**80** kHz

Förklaring:

- 1000 steg → **ARR = 1000 − 1 = 999**.
- Räknarklocka = 10 kHz × 1000 = 10 MHz. 80 MHz / 10 MHz = 8 → **PSC = 8 − 1 = 7**.
- Duty 35 % av 1000 = **350**.
- Max frekvens: PSC = 0 → 80 MHz / 1000 = **80 kHz** (över hörbart område ~20 kHz).

Formel: **f_PWM = f_clk / ((PSC+1)·(ARR+1))**

14. Husdjurssystem – marsvin (20 p)

Antaganden: Båda marsvinen är **inne** när systemet startar (`num_guinea_pigs_inside = 2`). LCD-drivrutin: `lcd_clear()`, `lcd_set_cursor(rad, kol)`, `lcd_print(text)`. ADC 12 bitar (0–4095).

a) I/O-pinnar (2 p)

Pinne	Inställning
PC0 (g0), PC1 (g1)	**GPIO_Input**. Ingen pull behövs (sensorn driver själv 0/3.3 V), gärna pull-down för säkerhets skull.
PA0	**Analog** – sätts upp av **ADC1, kanal IN0** (som uppgiften säger).
PC2	**GPIO_Output**, push-pull, start Low (summer).
PA8	**TIM1_CH1** – sätts upp av **TIM1** i "PWM Generation CH1". T.ex. PSC = 79 (1 MHz), ARR = 999 → 1 kHz PWM med 1000 steg.
PB6 / PB7	**I2C1_SCL / I2C1_SDA** – sätts upp av **I2C1**.

b) Tillståndsmaskin (7 p)

Idé: Ett marsvin som går **in** täcker sensorerna i ordningen: **g0 → g0+g1 → g1 → ingen**. Ut är tvärtom: **g1 → båda → g0 → ingen**. Vi följer den ordningen steg för steg. Om marsvinet vänder går vi tillbaka ett steg.

Tillstånd	Betyder	Nästa
IDLE	ingen sensor	g0 → IN_1, g1 → OUT_1
IN_1	bara yttre (på väg in)	båda → IN_2, ingen → IDLE
IN_2	båda	bara g1 → IN_3, bara g0 → IN_1
IN_3	bara inre	ingen → **inside++**, IDLE · båda → IN_2
OUT_1	bara inre (på väg ut)	båda → OUT_2, ingen → IDLE
OUT_2	båda	bara g0 → OUT_3, bara g1 → OUT_1
OUT_3	bara yttre	ingen → **inside−−**, IDLE · båda → OUT_2

```c
#include "main.h"
#include "lcd.h"
#include <stdio.h>

#define NUM_PIGS 2

extern ADC_HandleTypeDef hadc1;
extern TIM_HandleTypeDef htim1;

volatile int num_guinea_pigs_inside = NUM_PIGS;  // båda inne vid start

typedef enum { IDLE, IN_1, IN_2, IN_3, OUT_1, OUT_2, OUT_3 } pig_state_t;
static pig_state_t pig_state = IDLE;

void update_guinea_pig_state(void) {
```

```
  int g0 = (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_0) == GPIO_PIN_SET); // yttre
  int g1 = (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_1) == GPIO_PIN_SET); // inre

  switch (pig_state) {
  case IDLE:
    if (g0 && !g1)      pig_state = IN_1;
    else if (g1 && !g0) pig_state = OUT_1;
    break;

  /* ---- på väg IN ---- */
  case IN_1:
    if (g0 && g1)        pig_state = IN_2;
    else if (!g0 && !g1) pig_state = IDLE;     // vände
    break;
  case IN_2:
    if (!g0 && g1)       pig_state = IN_3;
    else if (g0 && !g1)  pig_state = IN_1;     // backar
    break;
  case IN_3:
    if (!g0 && !g1) {                          // gått in helt
      if (num_guinea_pigs_inside < NUM_PIGS) num_guinea_pigs_inside++;
      pig_state = IDLE;
    } else if (g0 && g1) pig_state = IN_2;
    break;

  /* ---- på väg UT ---- */
  case OUT_1:
    if (g0 && g1)        pig_state = OUT_2;
    else if (!g0 && !g1) pig_state = IDLE;
    break;
  case OUT_2:
    if (g0 && !g1)       pig_state = OUT_3;
    else if (!g0 && g1)  pig_state = OUT_1;
    break;
  case OUT_3:
    if (!g0 && !g1) {                          // gått ut helt
      if (num_guinea_pigs_inside > 0) num_guinea_pigs_inside--;
      pig_state = IDLE;
    } else if (g0 && g1) pig_state = OUT_2;
    break;
  }
}
```

Förklaring: Vi räknar bara när marsvinet gått **hela vägen** igenom. Om det bara tittar ut och vänder tillbaka går tillståndet tillbaka till IDLE utan att räkna. Gränserna (0 och 2) gör att räknaren aldrig blir fel om en sensor ger ett konstigt värde.

c) GetWaterLevel() (3 p)

```
float GetWaterLevel(void) {
  HAL_ADC_Start(&hadc1);                    // starta omvandling
  HAL_ADC_PollForConversion(&hadc1, 10);    // vänta (max 10 ms)
  uint32_t raw = HAL_ADC_GetValue(&hadc1);  // 0..4095
  HAL_ADC_Stop(&hadc1);
  return raw * 100.0f / 4095.0f;            // 0..100 %
}
```

Förklaring: 0 V → 0, 3.3 V → 4095. Dela med 4095 och gånga med 100 så får du procent. `100.0f` gör att uträkningen sker med flyttal (annars heltalsdivision!).

d) Mjuk tändning/släckning utan att blockera (3 p)

Svar: Använd **inte** `HAL_Delay` i en loop i 5 s (då missar vi sensorerna). Ha i stället ett **mål-värde** (0 eller 1000) och flytta CCR1 **ett steg åt gången**, med jämna mellanrum, från main-loopen (eller från ett timer-avbrott).

- 1000 steg på 5 s → **ett steg var 5:e ms**.
- Vi kollar med `HAL_GetTick()` om 5 ms har gått – om inte, går vi direkt vidare. Funktionen tar alltså nästan ingen tid.

```
void update_light(void) {
  static uint32_t last = 0;
  static uint32_t level = 0;              // nuvarande ljusstyrka 0..1000

  if (HAL_GetTick() - last < 5) return;   // inte dags än
  last = HAL_GetTick();

  int outside = NUM_PIGS - num_guinea_pigs_inside;
  uint32_t target = (outside > 0) ? 1000 : 0;   // på om någon är ute

  if (level < target)      level++;
  else if (level > target) level--;

  __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, level);  // TIM1->CCR1
}
```

Förklaring: Det här kallas **icke-blockerande** kod: varje anrop gör bara en liten bit av jobbet. Main-loopen hinner då läsa sensorerna tusentals gånger per sekund samtidigt som ljuset ändras mjukt.

e) Main-loopen (5 p)

```
int main(void) {
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  MX_ADC1_Init();
  MX_I2C1_Init();
  MX_TIM1_Init();

  HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);   // starta PWM för belysning
  __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 0);
  lcd_init();

  uint32_t last_lcd = 0;
  char buf[20];

  while (1) {
    update_guinea_pig_state();   // b) – körs så ofta som möjligt
    update_light();              // d) – mjuk dimning

    if (HAL_GetTick() - last_lcd >= 500) {   // var 0.5 s
      last_lcd = HAL_GetTick();

      float water = GetWaterLevel();        // c)

      // summer om vattnet är lågt
      HAL_GPIO_WritePin(GPIOC, GPIO_PIN_2,
                        (water < 20.0f) ? GPIO_PIN_SET : GPIO_PIN_RESET);
```

```
            lcd_clear();
            lcd_set_cursor(0, 0);
            sprintf(buf, "Inne: %d", num_guinea_pigs_inside);
            lcd_print(buf);
            lcd_set_cursor(1, 0);
            sprintf(buf, "Vatten: %d %%", (int)water);
            lcd_print(buf);
          }
        }
      }
```

Förklaring: Tillståndsmaskinen och belysningen körs **varje varv** i loopen (snabbt). LCD och vattennivå behöver bara uppdateras ibland (var 0.5 s), annars tar I2C-skrivningen onödigt mycket tid.